

GIL - Get in Line

Week 11 | From 4/21/2026 - 4/28/2026

Leader: Neel Vavilapalli

Disha Shetty ~29 hours Time Spent	Tasks Completed • Team Meeting • Designed and finalized the Get In Line logo — <i>~2 hours</i> • Hand-drew custom icons for each event type (capacity-based, table-based, attendance tracker), creating several different mockups — <i>~4-6 hours</i> • Created frame-by-frame animation for the UI, hand-drawn without any generative tools — <i>~5-8 hours</i> • Integrated custom icons and animation into the frontend, replacing generic lucide icons with branded assets — <i>~2 hours</i> • Formatted and polished UI layout to align with new visual assets — <i>~1.5 hours</i> • Added favicon to the web app — <i>~30 mins</i> • Fixed bugs related to past event table management, including table count reconciliation when switching between events — <i>~1.5 hours</i> • Integrated PostHog analytics into the frontend including SDK installation, environment variable configuration, and React provider setup and tested— <i>~4.5 hours</i> • Fixing an accidental breaking of code caused while teammate was modifying a function, requiring a full rollback to last stable deployment and manual re-doing of subsequent tasks-2.5 hrs • End to end testing of various user flows, during which the break was discovered. <i>~4 hrs</i> • Researched Vercel analytics, and looked into solution for our 2 bottlenecks: ○ Local storage limitations: IndexedDB ○ Single storage Supabase (inefficient): Redis Caching Challenges Faced • Git branch management caused repeated issues — a merge conflict in StaffDashboard.tsx resulted in the wrong version being pushed to main, requiring multiple force pushes to restore the correct codebase • Animation compatibility issues • PostHog integration required troubleshooting around empty environment variables and token mismatch — <i>~1 hour lost</i>
--	--

Angela Lopez Time Spent ~ 12 hours	Tasks completed: • Team Meeting • Ran tests from user side ◦ Joining capacity-based waitlists ◦ Joining table-based waitlists • Ran tests from staff side ◦ Creating/editing/deleting events ◦ Table-based ◦ Capacity-based ◦ Attendance-based • Table editing logic working correctly now ◦ Updates correctly with backend ◦ Table grid and table numbers update correctly ▪ Created and fixed bug that generated the wrong number of tables with the correct number of seats when tables were edited ▪ Fixed bug where original number of tables and its seats would pop back up after a few seconds even though they had already been edited ◦ Considers tables that are already occupied and does not allow edits until either table unoccupied or edit request is modified ▪ Sends an error message letting user know why they are unable to make edits • Fixed bugs in user view ◦ For queues, now shows current spots available and adjusts based on reservations made • Fixed text wrapping bug for event cards ◦ Was cutting off half of the event name • Changed attendance type events to always be private ◦ Before would show up in user view ◦ Now doesn't give toggle option to choose for the event to be public Any issues or challenges: • Had to start over with bug fixes because didn't realize it had broken other parts of the code
Neel Vavilapalli Time Spent : 10 hours	Tasks completed: • Team Meeting • Continued testing the application and identifying bugs and potential fixes to improve overall stability and performance.

	• Prepared for the mock presentation by organizing key points, refining slides, and practicing how to present the project clearly. • Reviewed the project report format and our current draft to ensure proper structure, completeness, and alignment with requirements. • Explored potential optimizations for the wait time prediction feature, focusing on improving accuracy and overall efficiency. • Tested backend AI endpoints to ensure calculations and outputs were accurate and consistent with expected results. Any issues or challenges: • Identifying source of bugs in project structure
Kyle Albuquerque Time Spent 11 hours	Tasks completed: • Team Meeting • Worked on Project Report (drafted sections, refined structure, and updated content for clarity) • Reviewed Specification Document to ensure alignment with project requirements • Researched potential future enhancements and scalability options for the project ◦ Multi-instance backend deployment with load balancing for true horizontal scalability ◦ Implementation of caching (e.g., Redis) to reduce database load and improve response times ◦ Real-time notifications ◦ Advanced analytics dashboard for businesses ◦ Role-based access control with more granular permissions for staff • Analyzed current system components to identify areas needing optimization ◦ Single-instance backend is a bottleneck ◦ Synchronization logic between localStorage and Supabase ◦ Database query performance and indexing within Supabase ◦ Lack of caching layer leading to repeated database calls ◦ Error handling • Evaluated performance bottlenecks and noted possible improvement strategies

	• Tested Business workflow for functionality, accuracy, and edge cases • Tested User workflow to ensure smooth user experience and proper system responses • Reviewed existing codebase/logic to better understand implementation details • Conducted basic validation of inputs/outputs across workflows Any issues or challenges: • Difficulty prioritizing which potential improvements would have the most impact ◦ Things that need optimizing also line up with future improvements
--	---

Total Time Spent: ~ 62 hours

Summary: This week, our team focused heavily on both frontend refinement and overall system stability. On the design side, we finalized the Get In Line logo and created fully custom, hand-drawn visual assets, including icons for different event types and a frame-by-frame UI animation. These assets were successfully integrated into the frontend, replacing generic icons and improving the overall look and feel of the application. We also polished the UI layout, added a favicon, and ensured the interface aligned with our updated branding. Alongside these improvements, we conducted extensive end-to-end testing across both user and staff workflows, covering features like joining waitlists, managing events, and handling attendance. Multiple bugs were identified and resolved, particularly around table-based event logic, UI display issues, and queue updates, resulting in a more reliable and consistent user experience.

In parallel, the team worked on backend validation, analytics integration, and project documentation. We integrated PostHog analytics into the frontend and tested backend API endpoints to verify the accuracy of wait time predictions. Additional effort went into debugging and recovering from a major code break caused by a merge conflict, which required rolling back to a stable version and carefully reapplying changes. We also explored performance and scalability improvements, including IndexedDB for local storage limitations and Redis caching to

reduce database load from Supabase. Beyond development, we made strong progress on the project report and mock presentation, refining structure, aligning with requirements, and preparing clear talking points. Overall, the week was centered on strengthening the application's reliability, improving its visual identity, and planning for future scalability and optimization.